# COMPARISON OF MACHINE LEARNING ALGORITHMS FOR NLP TASKS

**Objective:** Implement four machine learning algorithms for an NLP text-classification task and compare their performance. Sentiment analysis is used as the example task, where reviews are classified as Positive or Negative.

## 1. Understanding the Experiment

The experiment uses the same dataset and the same TF-IDF text features for four classifiers: Naive Bayes, Decision Tree, Random Forest and Logistic Regression. Each model is trained on the training portion and evaluated on unseen test reviews.

| Stage | Description |
|---|---|
| Input dataset | Read labelled text reviews from sentiment_dataset.csv. |
| Text representation | Use TF-IDF to convert review text into numerical feature vectors. |
| Model training | Train each of the four algorithms using the training data. |
| Prediction | Predict Positive or Negative sentiment for the test reviews. |
| Evaluation | Calculate accuracy, precision, recall and F1-score; inspect confusion matrices. |
| Comparison | Display a bar chart to compare the four algorithms. |

## 2. Software Requirements

Use Python with Jupyter Notebook, Anaconda, or a terminal. Install the required libraries:

```
pip install pandas numpy scikit-learn matplotlib seaborn
```

## 3. Dataset Preparation

Use the file **sentiment_dataset.csv** created for the previous experiment. It must contain two columns: **Review** (the text) and **Sentiment** (Positive or Negative). The supplied practice dataset contains 100 synthetic reviews: 50 positive and 50 negative.

| Review | Sentiment |
|---|---|
| The product is excellent | Positive |
| Amazing quality and fast delivery | Positive |
| Very poor quality | Negative |
| The service was terrible | Negative |

**Note:** The small synthetic dataset is suitable for demonstrating the workflow. It is not large or diverse enough to establish real-world model performance.

## 4. Complete Python Implementation

Save the following code as **compare_nlp_algorithms.py** in the same folder as **sentiment_dataset.csv**.

```
# COMPARISON OF MACHINE LEARNING ALGORITHMS FOR NLP TASKS

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline

from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
)

# STEP 1: Load dataset
data = pd.read_csv("sentiment_dataset.csv")
X = data["Review"]
y = data["Sentiment"]

# STEP 2: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# STEP 3: Define the four classifiers
models = {
    "Naive Bayes": MultinomialNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42
    ),
    "Logistic Regression": LogisticRegression(max_iter=1000)
}

results = []

# STEP 4: Train and evaluate each classifier
for name, model in models.items():
    print("\\nTraining:", name)

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(
            lowercase=True, stop_words="english"
        )),
        ("classifier", model)
    ])

    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(
        y_test, predictions, average="weighted", zero_division=0
    )
    recall = recall_score(
        y_test, predictions, average="weighted", zero_division=0
    )
    f1 = f1_score(
        y_test, predictions, average="weighted", zero_division=0
    )

    results.append({
        "Algorithm": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })

    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1-Score:", round(f1, 4))

    # Display confusion matrix
    cm = confusion_matrix(
        y_test, predictions, labels=["Negative", "Positive"]
    )

    plt.figure(figsize=(5, 4))
    sns.heatmap(
        cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=["Negative", "Positive"],
```

```
        yticklabels=["Negative", "Positive"]
    )
    plt.title(name + " - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.show()

# STEP 5: Display final comparison table
results_df = pd.DataFrame(results)
print("\\nFINAL ALGORITHM COMPARISON")
print(results_df.to_string(index=False))

# STEP 6: Plot performance comparison
results_df.set_index("Algorithm")[
    ["Accuracy", "Precision", "Recall", "F1-Score"]
].plot(kind="bar", figsize=(10, 5))

plt.title("Comparison of Machine Learning Algorithms")
plt.ylabel("Performance Score")
plt.xlabel("Algorithms")
plt.ylim(0, 1.1)
plt.xticks(rotation=20)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

## 5. How to Execute the Program

**1.** Create a folder named **NLP_Algorithm_Comparison**.

**2.** Place **sentiment_dataset.csv** and **compare_nlp_algorithms.py** inside that folder.

**3.** Open Command Prompt or a terminal and navigate to the project folder.

**4.** Install the libraries using the pip command in Section 2.

**5.** Run the Python script and wait for all four models to train and evaluate.

**6.** Observe the printed metrics, four confusion matrices and final comparison chart.

## Command Prompt commands

```
cd path\\to\\NLP_Algorithm_Comparison\npip install pandas numpy scikit-learn matplotlib seaborn\npython compare_nlp_algorithms.py
```

## 6. Understanding the Four Algorithms

| Algorithm | Basic working principle |
|---|---|
| Naive Bayes | Uses Bayes' theorem and word-related probabilities to estimate the class of a review. |
| Decision Tree | Learns a sequence of feature-based decision rules to classify a review. |
| Random Forest | Combines predictions from many decision trees to produce a final class. |
| Logistic Regression | Estimates class probabilities using a learned linear decision function. |

## 7. Understanding Evaluation Metrics

| Metric | Meaning |
|---|---|
| Accuracy | Proportion of all test examples classified correctly. |
| Precision | Of the examples predicted as a class, the proportion that truly belongs to that class. |
| Recall | Of the actual examples in a class, the proportion correctly identified. |
| F1-score | Harmonic mean of precision and recall. |

## 8. Understanding the Output

The program displays a classification report summary through the four calculated metrics, a confusion matrix for each model, a final comparison table and a grouped bar chart. The confusion matrix shows the number of correct and incorrect predictions for each class. The bar chart helps compare the metric values across algorithms.

## 9. Suggested Laboratory Experiment

**Title:** Comparison of Machine Learning Algorithms for NLP Tasks

**Aim:** To implement and compare Naive Bayes, Decision Tree, Random Forest and Logistic Regression algorithms for sentiment classification using TF-IDF feature extraction, and evaluate their performance using accuracy, precision, recall and F1-score.

**Learning Outcomes**

1. Understand the application of machine learning algorithms in NLP.
2. Implement four classification algorithms for sentiment analysis.
3. Evaluate classification performance using standard metrics.
4. Compare and interpret the results of different algorithms.

**Important teaching point:** TF-IDF is the common feature extraction technique used to represent the text numerically before classification. In the provided code, each model is placed in a pipeline so that TF-IDF is fitted only on the training data, helping prevent test-data leakage. The current 100-review synthetic dataset is intended for practice; larger and more diverse datasets are needed for reliable conclusions.